Infinite Connections

Frequently Asked Questions

Abuzar | Adreama | Burak | Hengkai | Kaiwen
STA 561D | Duke University | April 2026

This FAQ anticipates the questions a careful, skeptical reader would ask after reading our executive summary or technical report. We group them by topic: how each of the five teammates' methods works, how we prevent duplicates, how we measure quality, cost and scale, and known limitations.

The five methods, explained individually

Q. What method did Kaiwen build?

A. Kaiwen implemented two pipelines head-to-head. Pipeline A is a reference implementation of the iterative generator from the 'Making New Connections' paper -- five AI calls per puzzle, one for each of the four groups plus a final editor pass. Pipeline B is a new design called CATEGORY-FIRST RETRIEVAL (CFR): instead of asking the AI to produce words, it asks the AI only for CATEGORY NAMES, and then a word-lookup against a 14,877-word dictionary finds the best matching words. Mode A of CFR uses zero AI calls (it samples category names from the existing NYT catalog); Mode B uses one AI call per puzzle to invent fresh category names. One AI call replaces five at the same quality.

Q. What method did Adreama build?

A. Adreama's method is a single-call iterative generator. A GPT-4.1 call returns all four groups at once as structured JSON, with a long context prompt that carries forward every word and category the generator has used across sessions. The generator keeps three persistent memory files (used_words.json, used_categories.json, used_boards.json) so duplicates never creep in even over thousands of puzzles. A library of 48 hand-curated 'category webs' (tuples of REAL + TRAP categories) seeds thematic false-group designs. The generator retries up to four times on parse or validation errors; batches of 50 attempts typically yield ~35 valid puzzles.

Q. What method did Burak build?

A. Burak's method is almost entirely non-AI -- the AI only gets called to label one group. The pipeline builds puzzles color by color, each color using a different algorithm: PURPLE is drawn from a pool scored by eight rule-based mechanisms (e.g., collective nouns, contronyms), optionally with a machine-learning classifier reranking the pool; GREEN is built by rhyme anchors using the CMU pronunciation dictionary; BLUE is a niche-category generator where Claude Haiku validates the group; and YELLOW uses word2vec semantic clusters. 'Impostor' words are planted deliberately to create false-group pressure across colors.

Q. What method did Hengkai build?

A. Hengkai is the yellow-difficulty specialist. The generator is fully algorithmic -- zero AI calls -- and uses pre-computed GloVe word embeddings, ConceptNet relations, and corpus frequency filters. Candidates are scored within several relation types (taxonomic, synonymy, verb-noun association, ConceptNet link) and the highest-scoring combinations become yellow groups. It is the only teammate's generator that validates each candidate group against the full 554-puzzle NYT history before acceptance, rejecting any group that shares three or more words with a past NYT group. A checkpoint system lets batch runs of 700+ puzzles resume after interruption.

Q. What method did Abuzar build?

A. Abuzar's method is an agent-oriented system. A single GPT-4.1 call produces the full 4-group puzzle in JSON, but the prompt is heavily engineered with banned-category lists, banned-purple-type counters, and a trap-word mechanism where one purple word is specifically designed to impersonate a simpler category. The pipeline enforces self-uniqueness through persistent word, category, and board-signature sets; it also ships with a Flask web server and a 'concept inspirations' library for theme variety. It does NOT check against the 554 past NYT puzzles -- only against its own generated history.

Q. Why build five different methods instead of picking the best one?

A. The Connections puzzle is deceptively hard. A single method tends to be great at one category style and weak at others: semantic-similarity approaches nail 'SYNONYMS FOR X' but stumble on wordplay like '___FISH' or hidden patterns like 'WORDS CONTAINING COLORS.' By spreading the work across five complementary methods we cover

more of the design space than any one approach could. It also matches the professor's suggestion that we IDENTIFY THE DIFFERENT MECHANISMS IN WHICH WORDS ARE GROUPED AND BUILD SEPARATE GENERATORS FOR EACH.

Q. Why use embeddings at all -- aren't LLMs just as good?

A. LLMs are good at creative naming but inconsistent at ranking. Asked to pick 'the best 4 of these 8 words,' an LLM gives different answers each time depending on temperature, seed, and context. Embeddings (numerical fingerprints of each word's meaning) give us DETERMINISTIC similarity scores: we get the same answer every time, which makes difficulty colors and validation reproducible. The combined approach -- LLM for naming, embeddings for ranking -- gets the best of both.

Preventing duplicates

Q. How does each teammate prevent duplicates against past NYT puzzles?

A. Not all five methods check against the 554 past NYT puzzles. Here is the honest breakdown:

Teammate	Check vs past NYT puzzles?	Self-duplicate check
Kaiwen	YES: 16-word overlap>6 + exact 4-word group match	implicit, per-batch
Hengkai	YES: reject if >=3 words overlap past NYT groups	anchor-repeat limit + checkpoints
Adreama	No explicit check vs NYT	YES: persistent JSON files
Abuzar	No explicit check vs NYT	YES: persistent JSON files
Burak	No explicit check vs NYT	in-memory only (per notebook run)

The strongest guarantee comes from Kaiwen's pipeline, which enforces the check both during word selection (skip-and-retry) and as a final gate. Because Kaiwen's roundtable validator is used as the shared acceptance step across the team's final merged dataset, any puzzle that slips through a teammate's own generator still has to pass the 4-word-group match before it lands in the submitted dataset. In that sense the team-wide submitted dataset IS guarded against NYT duplication -- via Kaiwen's step -- even when the upstream generator didn't check.

Q. What about self-duplication across batches?

A. Four of the five methods have an explicit self-duplication defense. Adreama and Abuzar persist three files each (used_words.json, used_categories.json, used_boards.json) that carry state across sessions. Hengkai's batch runner checkpoints to a JSON file and resumes without repeating groups. Kaiwen's CFR excludes already-used words within a puzzle and deduplicates by board signature across batches. Burak's notebook, the exception, holds its dedup state only in memory -- a repeated notebook run can in principle regenerate the same puzzle. In practice our merged dataset is deduplicated by board signature as a final step, which removes any cross-teammate collisions.

Q. What about near-duplicates -- a puzzle with 5 overlapping words?

A. Kaiwen's threshold is 6 (any overlap of 7 or more words flags the puzzle; Hengkai's threshold for yellow groups alone is 3). With 16-word puzzles drawn from a 14,877-word bank, overlapping 5 or 6 words happens by chance perhaps once in thousands of generated puzzles. The per-puzzle metadata we save includes the overlap count against each past NYT puzzle so a reviewer can audit this directly and adjust the threshold if desired.

Q. Could the LLM leak a memorized NYT puzzle?

A. The underlying language models were trained on web text, so they have certainly seen Connections discussions. Three of our pipelines (Kaiwen, Burak, Hengkai) do not let the LLM produce the final word list directly -- they only let it propose category names or bless a candidate group, while the actual words come from deterministic dictionary lookup. For the pipelines that do ask the LLM to emit the full 16 words (Adreama and Abuzar), the group-level dedup in our merged pipeline catches any leaked group before acceptance.

Quality and evaluation

Q. 99% pass rate -- doesn't that mean your solver is too lenient?

A. We don't trust a single solver; that's why Kaiwen's pipeline runs TWO independent ones. A puzzle is accepted only if at least one solver fully recovers the intended 4-group partition. Both solvers use deterministic algorithms (greedy cosine selection and beam-search clustering) and solve only 2-3% of the real NYT puzzles when tested directly on them -- they

are conservative, not easy. So a puzzle that our solvers CAN crack has a cleanly recoverable structure (exactly what a good puzzle should have).

Q. How close to the NYT style are the generated puzzles?

A. We have not yet run a formal human evaluation. Internally, on the categories where our methods excel (synonyms, types-of-X, themed nouns) many puzzles are difficult to distinguish from NYT output on casual inspection. On pure wordplay categories ('___FISH,' letter homophones), the output is weaker -- see the limitations section. We added a thumbs-up/thumbs-down rating button to the web app so real players can rate puzzles at scale, and we plan to collect at least several hundred ratings before submission.

Q. What's the unique-solution guarantee?

A. A Connections puzzle is only valid if exactly one 4-way partition fits the intended themes. Our pipeline enforces this in two ways: (1) during generation, CFR's candidate selector skips any 4-word group with a high cross-group 'bleed' score (words that fit another group too well), and (2) during validation, our two solvers must find the intended partition -- if they find a DIFFERENT partition that also satisfies the themes, the puzzle is rejected as ambiguous.

Q. Is the word bank big enough?

A. The NYT has used about 4,918 unique words across all 554 of its puzzles. Kaiwen's augmented bank has 14,877 WORDS -- three times larger. In the CFR output, 62-69% of generated words come from the NON-NYT portion, showing the expansion is actually being used. Burak and Hengkai additionally pull from WordNet, ConceptNet, and the Brown corpus, which expand the vocabulary further within those specialized pipelines.

Cost and scale

Q. What does it cost to generate 10,000 puzzles with each method?

A. Approximate cost per 10,000 puzzles:

Method	Approx cost / 10,000 puzzles
Kaiwen CFR Mode A (no AI calls)	$0
Kaiwen CFR Mode B (1 AI call each)	~$3
Hengkai yellow generator (no AI calls)	$0
Burak pattern-builder (2 AI calls each)	~$20
Adreama iterative (1 long AI call each)	~$10
Abuzar agent system (1 long AI call)	~$15
Pipeline A baseline (5 AI calls each)	~$50

The cheapest methods let us generate unlimited puzzles at zero cost; even the most expensive is within a routine student budget.

Q. Why are different methods cheaper than others?

A. They make fewer calls to the paid AI service. Pipeline A asks the AI five times per puzzle. Adreama and Abuzar each make one long call. CFR Mode B makes one short call. CFR Mode A and Hengkai's generator make zero calls -- they use dictionaries, embeddings, and math, which run free on a laptop.

Q. Will this scale to 100,000 puzzles?

A. Yes. CFR Mode A runs at about 1.3 seconds per puzzle on a single laptop CPU with no network calls, so 100,000 puzzles take about 36 hours of wall time and cost $0. Hengkai's batch generator scales similarly. The bottleneck at that scale is the roundtable validator, not generation.

Reproducibility

Q. Can someone else reproduce your results without API keys?

A. Mostly, yes. Kaiwen's code ships with a DRY_RUN=true default: every call to the AI service is intercepted and replaced by a realistic mock response. The master notebook, the web app, the test suite, and the solver benchmark all run end-to-end with no keys configured. Setting DRY_RUN=false (with OPENAI_API_KEY) switches to real calls. Adreama's and Abuzar's generators require live API access. Burak's and Hengkai's generators run offline after pre-computing their embeddings and corpora.

Q. How do I re-run the benchmarks?

A. Two commands for Kaiwen's pipeline:

```
python scripts/cfr/generate_cfr.py --mode remix --count 100
python scripts/cfr/generate_cfr.py --mode fresh --count 100
```

Output is written to data/generated/cfr/... with per-puzzle metadata (pass rate, solver traces, dedup results) for audit. Other teammates' generators live in their own folders and each ship with instructions.

Q. Where is the code?

A. The primary repository is at https://github.com/Kevin2330/nyt-connections-generator. The live web app at kevin2330.github.io/nyt-connections-generator/ loads 540+ validated puzzles from the same repo. Individual teammate code lives in the STA561/ directory of the submission; Abuzar's code additionally lives in a private team repository.

Limitations

Q. What's the biggest weakness of the overall system?

A. Wordplay and syntactic categories. Our embedding-based methods (Kaiwen, Hengkai) rely on semantic similarity, which captures 'SYNONYMS FOR SMART' but not 'WORDS THAT END IN -ING' or '___FISH.' These pattern-based categories require string-level and phonetic reasoning that embeddings don't naturally do well. Burak's CMU-rhyme green generator and Abuzar's multi-agent system partly address this, but the hardest (purple) categories remain our collective weakest output.

Q. Why does WordNet sometimes return awkward words (e.g., MORPHOPHONEMIC)?

A. WordNet includes many archaic and technical words. Kaiwen's pipeline filters by corpus frequency and character length, but some oddities slip through. A higher frequency threshold or intersection with a common-English list (the Google 10,000-most-common-words list is already included as an optional filter) would tighten this further.

Q. Why does Burak's generator have no cross-session deduplication?

A. It is an honest gap. Burak's notebook holds its used-words set only in memory, so two separate notebook runs could in principle generate the same puzzle. In practice the final merged dataset is deduplicated by 16-word board signature as a post-processing step, which removes any such collisions before submission.

Teamwork

Q. Who did what?

A.

Teammate	Contribution
Adreama	Iterative AI generator with cross-session memory and trap-word webs
Burak	Non-AI pattern-driven builder (purple mechanism + rhyme green + word2vec yellow)
Hengkai	Yellow specialist: embedding + ConceptNet scoring, NYT-history overlap check
Kaiwen	Pipeline A baseline, CFR Modes A & B, roundtable validator, web app
Abuzar	Multi-agent critic system + Flask web server + concept-inspiration library